Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
Университетский технологический колледж
Кафедра Информационные системы в экономике

Отчет защищен с оценкой _____

Преподаватель С.В.Умбетов

«___» _____ 2026 г.

Отчет по лабораторной работе №
по дисциплине «Основы компьютерных сетей»
«Анализ сетевых портов и активных соединений»

ЛР 09.02.08.19.002

Студент группы 1ИИС-41 Н.Ю. Потапов
группа инициалы., фамилия
Преподаватель старший преподаватель, доцент С.В. Умбетов
должность, ученая степень инициалы., фамилия

Барнаул 2026

Цель работы: Изучение механизмов работы сетевых портов и сокетов в операционной системе Linux, освоение утилит анализа активных соединений и слушающих портов: ss, netstat и lsof. Студенты научатся запускать простейшие сетевые службы, определять, какие порты открыты, какие процессы их занимают, выявлять конфликты портов и анализировать состояния TCP-соединений (LISTEN, ESTABLISHED, TIME_WAIT). Работа выполняется на Ubuntu Server в VirtualBox и на локальной машине для сравнения результатов.

Задачи к работе:

- С помощью утилиты ss вывести список всех слушающих портов на Ubuntu Server и определить, какие службы соответствуют обнаруженным портам (22, 53 и другим).
- Установить пакет net-tools и сравнить вывод команды netstat -tulnp с выводом ss. Подробно описать различия между утилитами.
- Запустить простой HTTP-сервер на Python (python3 -m http.server 8000) и с помощью ss и lsof подтвердить, что порт 8000 находится в состоянии LISTEN и принадлежит процессу python.
- Смоделировать конфликт портов, запустив второй HTTP-сервер на том же порту 8000. Зафиксировать сообщение об ошибке, найти PID первого процесса, завершить его и успешно запустить сервер снова.
- Запустить netcat в режиме прослушивания на порту 9999, проверить появление порта через ss/lsof, подключиться к нему с другого терминала и зафиксировать изменение состояния сокета.
- Вывести все ESTABLISHED-соединения, найти среди них SSH-подключение и записать локальный и удалённый адрес/порт.
- (Дополнительно) С помощью ss или netstat посчитать количество сокетов в состоянии TIME_WAIT и подробно объяснить механизм их возникновения.
- Ответить на контрольные вопросы по теме лабораторной работы.

Задание принял:  Потапов Н.Ю.

Подпись ФИО

Ход работы:

Задание 1. Просмотр слушающих портов с помощью ss

Ход выполнения: Выполнил команду sudo ss -tulnp на Ubuntu Server для вывода списка всех слушающих TCP и UDP портов с указанием процессов. Ключи команды: -t (TCP), -u (UDP), -l (только слушающие), -n (не резолвить имена), -p (показать PID и имя процесса).

В выводе были обнаружены следующие слушающие порты:

- Порт 22 (TCP) — процесс sshd (PID указан в выводе). Это SSH-сервер, обеспечивающий удалённое подключение к серверу. Порт входит в диапазон привилегированных (0–1023) и требует прав root для прослушивания.

- Порт 53 (UDP/TCP) — процесс systemd-resolved. Это локальный DNS-резолвер, обслуживающий запросы на разрешение доменных имён.

- Порт 68 (UDP) — процесс systemd-networkd или dhclient. DHCP-клиент, получающий IP-адрес от виртуального адаптера VirtualBox.

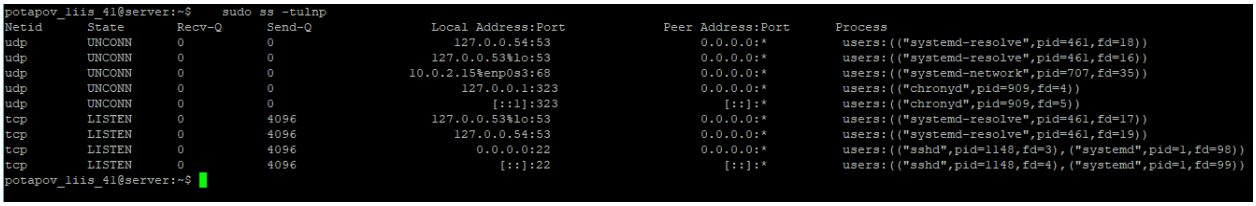


Рисунок 1. Выполнение команды sudo ss -tulnp на Ubuntu Server.

Задание 2. Сравнение утилит ss и netstat

Ход выполнения: Установил пакет net-tools командой sudo apt install net-tools -y, после чего выполнил аналогичную команду sudo netstat -tulnp. Визуально вывод обеих команд похож: отображаются протокол, состояние, локальный и удалённый адрес, PID/имя процесса. Однако между утилитами существуют принципиальные различия.

Подробное сравнение:

1. Источник данных. Утилита ss входит в пакет iproute2 и получает информацию напрямую из ядра Linux через netlink-сокеты. Утилита netstat (пакет net-tools) читает текстовые файлы в виртуальной файловой системе /proc/net/ (например, /proc/net/tcp, /proc/net/udp).

2. Производительность. На системах с большим числом соединений (тысячи и десятки тысяч) ss работает значительно быстрее, так как не выполняет парсинг текстовых файлов. netstat в таких условиях может работать медленно и потреблять больше ресурсов.

3. Функциональность. ss поддерживает более гибкие фильтры (например, sport = :80, dport = :443, фильтрация по состоянию), тогда как netstat требует дополнительной обработки через grep.

4. Актуальность. netstat считается устаревшей утилитой и не развивается. В современных дистрибутивах Linux (включая Ubuntu Server) рекомендуется использовать ss.

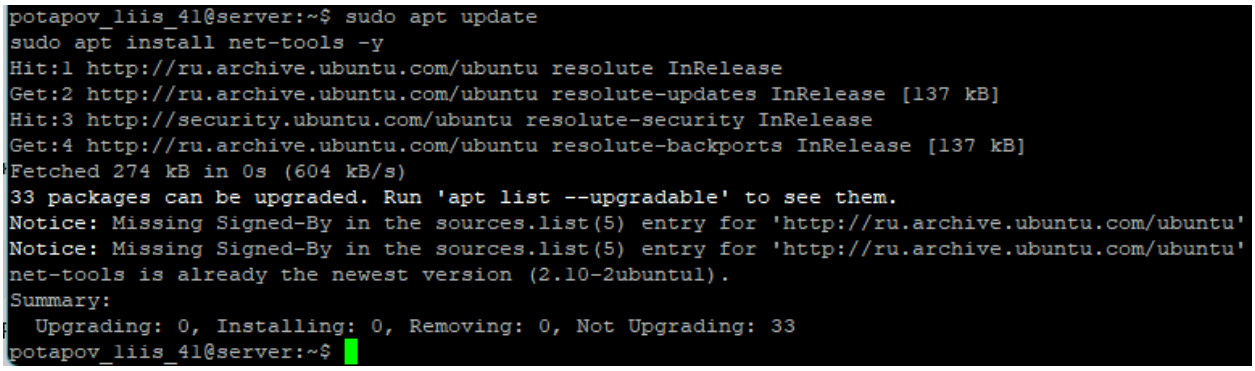


Рисунок 2. Net-tools успешно установлен



Рисунок 3. Выполнение команды sudo netstat -tulnp на Ubuntu Server.

Задание 3. Запуск HTTP-сервера и проверка порта

Ход выполнения: В первом терминале запустил простой HTTP-сервер на Python командой python3 -m http.server 8000. Сервер успешно стартовал и перешёл в режим ожидания подключений.

Во втором терминале выполнил команды для проверки состояния порта:

- sudo ss -tlnp | grep 8000 — показал, что порт 8000 находится в состоянии LISTEN и принадлежит процессу python3 с соответствующим PID.

- sudo lsof -i :8000 — подтвердил, что процесс python3 открыл TCP-сокет на порту 8000 в режиме прослушивания.

Обе утилиты дали согласованный результат: порт 8000 занят процессом python, сокет находится в состоянии LISTEN, что означает готовность сервера принимать входящие HTTP-запросы.

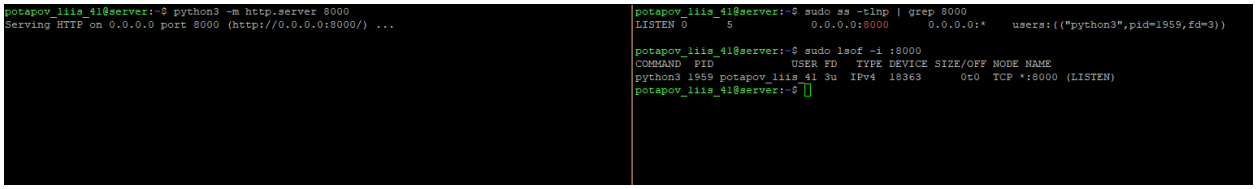


Рисунок 4. Запуск python3 -m http.server 8000 и проверка порта через ss и lsof.

Задание 4. Моделирование и разрешение конфликта портов

Ход выполнения: Пока первый экземпляр HTTP-сервера продолжал работать, в третьем терминале попытался запустить второй экземпляр той же командой python3 -m http.server 8000.

Зафиксированная ошибка: OSError: Address already in use

Обоснование ошибки: Операционная система не позволяет двум процессам одновременно слушать один и тот же TCP-порт на одном сетевом интерфейсе. Это ограничение связано с тем, что при поступлении входящего пакета на порт 8000 сетевой стек ядра не сможет однозначно определить, какому из двух процессов его доставить. Такое поведение является защитным механизмом, предотвращающим конфликты между сетевыми службами.

Разрешение конфликта:

1. С помощью команды sudo lsof -i :8000 определил PID первого процесса python (например, PID = 2472).

2. Завершил процесс командой kill 2472. В первом терминале сервер остановился.

3. Повторно выполнил python3 -m http.server 8000 — на этот раз сервер запустился успешно, так как порт был освобождён.

Альтернативный способ разрешения конфликта — запустить вторую службу на другом свободном порту (например, 8001), не завершая первую.

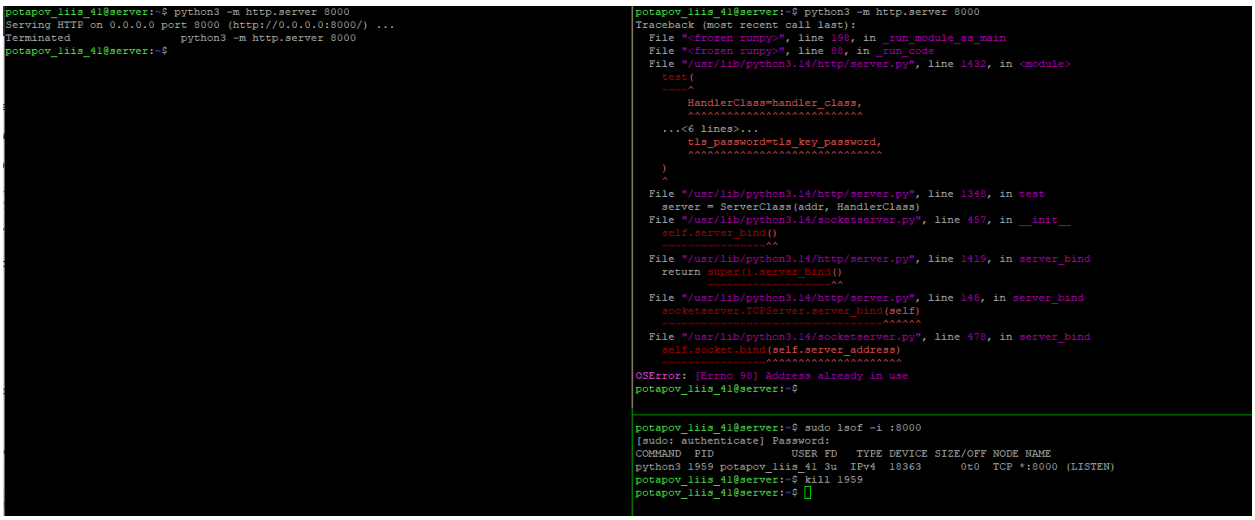


Рисунок 5. Ошибка «Address already in use» при попытке запуска второго сервера, поиск PID через lsof, завершение процесса и успешный перезапуск сервера.

Задание 5. Работа с netcat и изменение состояния сокета

Ход выполнения: В первом терминале запустил netcat в режиме прослушивания командой nc -l -p 9999 (в некоторых версиях netcat используется синтаксис nc -l 9999).

Во втором терминале проверил состояние порта:

- sudo ss -tlnp | grep 9999 — показал, что порт 9999 находится в состоянии LISTEN и принадлежит процессу nc.

Затем в третьем терминале подключился к серверу netcat командой nc 127.0.0.1 9999 (или по IP-адресу сервера 10.0.2.15). После установления соединения ввёл тестовое сообщение, которое успешно отобразилось в первом терминале.

После подключения повторно проверил состояние сокета командой sudo ss -tnp | grep 9999 (без флага -l, чтобы видеть все соединения). Результат показал, что сокет перешёл из состояния LISTEN в состояние ESTABLISHED. В выводе появились две строки:

- Серверная сторона: 127.0.0.1:9999 -> 127.0.0.1:<эфемерный_порт> (ESTABLISHED)
- Клиентская сторона: 127.0.0.1:<эфемерный_порт> -> 127.0.0.1:9999 (ESTABLISHED)

Обоснование изменения состояния: До подключения сокет находился в состоянии LISTEN — сервер ожидал входящих соединений. После успешного завершения TCP handshake (трёхэтапного рукопожатия: SYN → SYN-ACK → ACK) сокет перешёл в состояние ESTABLISHED, что означает активное установленное соединение, готовое к обмену данными.

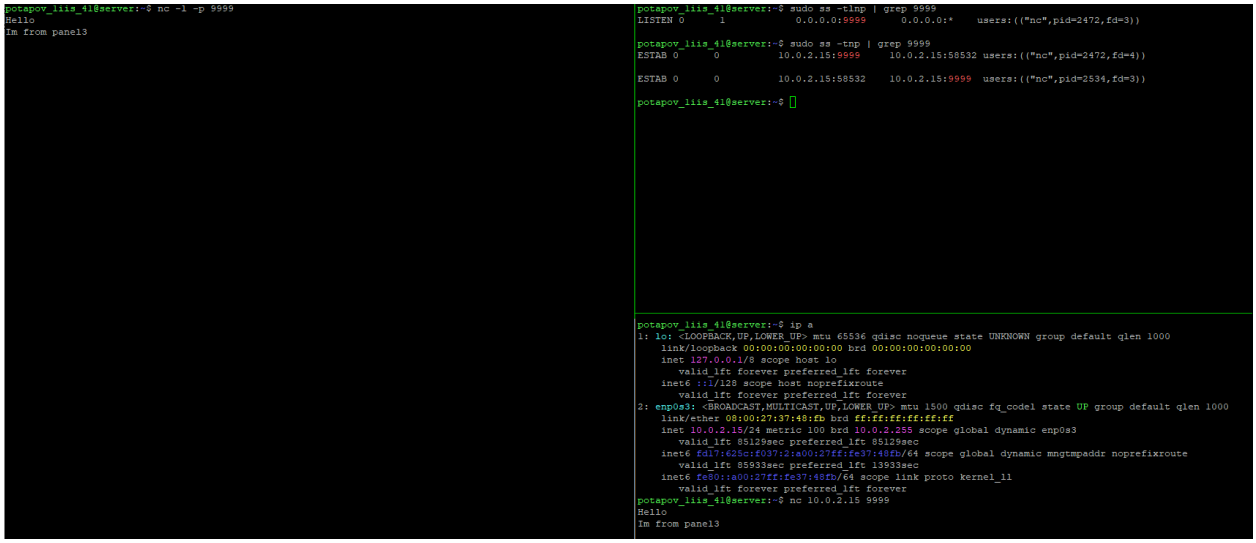


Рисунок 6. Запуск nc -l -p 9999, подключение клиента nc и переход сокета в состояние ESTABLISHED.

Задание 6. Просмотр активных (ESTABLISHED) соединений

Ход выполнения: Выполнил команду sudo lsof -i TCP -s TCP:ESTABLISHED для вывода всех активных TCP-соединений.

В результате были обнаружены следующие соединения:

1) SSH-подключение (мой текущий сеанс):
    a) Локальный адрес: server:22 (порт SSH, привилегированный порт)
    b) Удалённый адрес: _gateway:55518 (шлюз VirtualBox 10.0.2.2 с эфемерным портом 55518)
    c) Процесс: sshd-sess, пользователь potapov_liis_41
    d) Состояние: ESTABLISHED — активное соединение, идёт обмен данными
2) Netcat-соединение (из задания 5):
    a) Серверная сторона: server:9999 -> server:58532, PID процесса nc
    b) Клиентская сторона: server:58532 -> server:9999, PID процесса nc

c) Порт 58532 — эфемерный порт, автоматически назначенный ОС клиентскому процессу

Анализ: SSH-соединение использует привилегированный порт 22 на стороне сервера и эфемерный порт 55518 на стороне клиента (шлюза VirtualBox). Netcat-соединение полностью локальное (127.0.0.1), сервер слушает порт 9999, клиент использует эфемерный порт 58532.

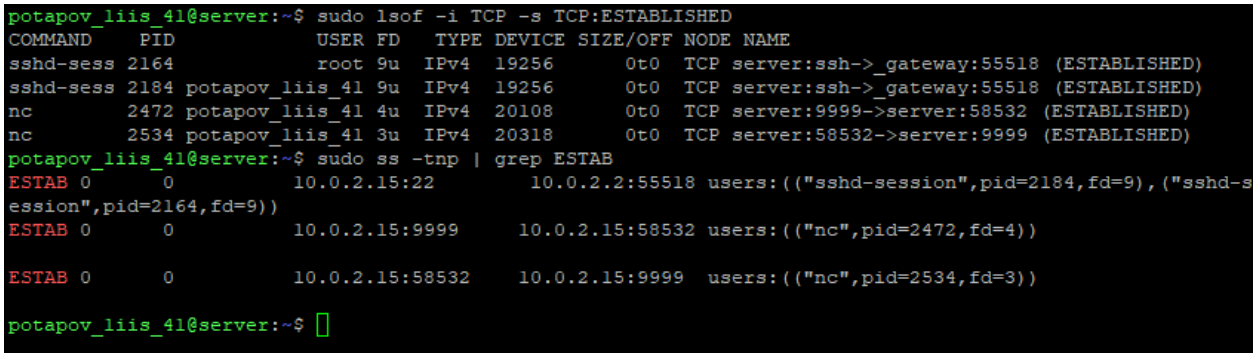


Рисунок 7. Вывод команды lsof и ss с активными соединениями.

Задание 7. Подсчёт сокетов в состоянии TIME_WAIT

Ход выполнения: Выполнил команду для подсчёта сокетов в состоянии TIME_WAIT: sudo ss -tan | grep TIME-WAIT | wc -l

Подробное объяснение механизма TIME_WAIT:

Состояние TIME_WAIT возникает на стороне, которая первой инициировала активное закрытие TCP-соединения (отправила первый FIN-пакет). После того как обе стороны обменялись FIN и ACK пакетами, сокет переходит в состояние TIME_WAIT и остаётся в нём в течение определённого времени (обычно 60 секунд, что соответствует 2 × MSL — Maximum Segment Lifetime).

Причины существования TIME_WAIT:

1. Гарантия доставки последнего ACK. Если последний ACK-пакет, подтверждающий закрытие соединения, будет потерян в сети, удалённая сторона повторно отправит FIN-пакет. Сокет в состоянии TIME_WAIT сможет корректно обработать этот повторный FIN и отправить ACK ещё раз.

2. Защита от «блуждающих» пакетов. В сети могут находиться пакеты от предыдущего соединения, которые задержались из-за маршрутизации или перегрузок. Если сразу после закрытия соединения открыть новое с теми же адресами и портами, задержавшиеся пакеты могут быть ошибочно приняты за пакеты нового соединения. TIME_WAIT предотвращает такую ситуацию, выдерживая паузу, достаточную для «рассасывания» старых пакетов в сети.

3. Последствия большого числа TIME_WAIT. На высоконагруженных серверах (например, веб-серверах с короткими соединениями) может накапливаться большое количество сокетов в TIME_WAIT, что приводит к исчерпанию доступных портов. Для решения этой проблемы применяют настройку tcp_tw_reuse в ядре Linux или используют keep-alive соединения.

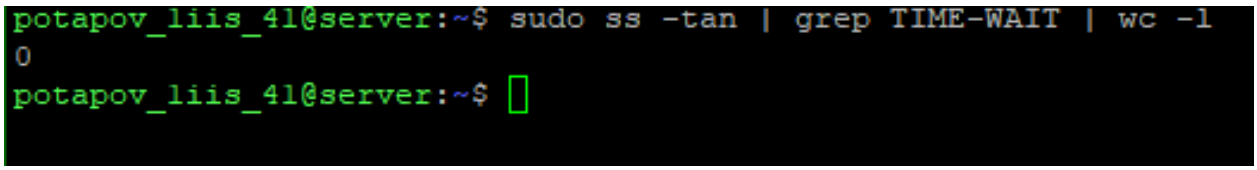


Рисунок 8. Подсчёт сокетов в состоянии TIME_WAIT командой ss.

Вывод:

В ходе лабораторной работы я изучил механизмы работы сетевых портов и сокетов в операционной системе Linux и освоил утилиты анализа активных соединений: ss, netstat и lsof. На практике научился выводить список слушающих портов и идентифицировать соответствующие им службы (SSH на порту 22, DNS на порту 53, DHCP на порту 68). Освоил запуск простейших сетевых служб (HTTP-сервер Python, netcat) и наблюдение за состоянием их сокетов. Получил опыт моделирования и разрешения конфликтов портов: зафиксировал ошибку «Address already in use», научился находить PID процесса через lsof и завершать его командой kill. Изучил переход сокета из состояния LISTEN в ESTABLISHED при установлении TCP-соединения. Проанализировал активные соединения, обнаружил своё SSH-подключение и локальное netcat-соединение, определил эфемерные порты клиентов. Также разобрался с механизмом состояния TIME_WAIT и его ролью в надёжном закрытии TCP-соединений. Приобретённые навыки позволяют эффективно диагностировать сетевые проблемы, выявлять занятые порты и управлять сетевыми службами в Linux.

Ссылка на мой репозиторий с работами

Ответы на контрольные вопросы:

1. Что такое сетевой порт и зачем он нужен?

Сетевой порт — это 16-битное число (от 0 до 65535), которое вместе с IP-адресом и протоколом (TCP или UDP) образует сокет — конечную точку сетевого соединения. Порт нужен для того, чтобы на одном IP-адресе могло одновременно работать несколько сетевых служб. Например, веб-сервер слушает порт 80, SSH-сервер — порт 22, DNS — порт 53. Без портов операционная система не смогла бы определить, какому приложению доставить входящий сетевой пакет.

2. Какие диапазоны портов существуют и чем они отличаются?

Существуют три диапазона портов:

0–1023 — привилегированные (well-known). Используются системными службами (HTTP 80, HTTPS 443, SSH 22, DNS 53). Для прослушивания требуются права root.

1024–49151 — зарегистрированные (registered). Назначаются IANA для конкретных приложений (например, MySQL — 3306, PostgreSQL — 5432).

49152–65535 — динамические/эфемерные. Автоматически назначаются операционной системой клиентским приложениям на время исходящего соединения.

3. Что означает состояние LISTEN у TCP-сокета?

Состояние LISTEN означает, что сокет ожидает входящих соединений. Служба запущена,-bind() и listen() выполнены успешно, и сервер готов принимать клиентские подключения. В этом состоянии сокет не связан с конкретным клиентом — он «слушает» порт и ждёт первого SYN-пакета для начала TCP handshake.

4. Чем состояние ESTABLISHED отличается от TIME_WAIT?

ESTABLISHED — активное, открытое TCP-соединение, по которому идёт обмен данными. Обе стороны подтвердили установление соединения (завершён handshake).

TIME_WAIT — соединение уже закрыто (обе стороны обменялись FIN/ACK), но сокет временно зарезервирован операционной системой. Сокет находится в

этом состоянии обычно 60 секунд (2 × MSL) для гарантии доставки последнего ACK и предотвращения попадания «блуждающих» пакетов от старого соединения в новое.

5. Почему утилита ss предпочтительнее netstat?

Утилита ss входит в современный пакет iproute2 и получает данные напрямую из ядра Linux через netlink-сокеты, что обеспечивает высокую производительность, особенно на системах с большим числом соединений. netstat (пакет net-tools) читает текстовые файлы в /proc/net/, что медленнее и неэффективно. Кроме того, ss поддерживает более гибкие фильтры и активно развивается, тогда как netstat считается устаревшей и не получает новых возможностей.

6. Какие ключи ss позволяют увидеть слушающие порты вместе с процессами?

Команда: sudo ss -tulnp

-t — показать TCP-сокеты

-u — показать UDP-сокеты

-l — только слушающие (LISTEN)

-n — не резолвить имена (показывать числовые порты)

-p — показать PID и имя процесса (требуются права root)

7. Как с помощью lsof узнать, какой процесс занимает порт 8080?

Команда: sudo lsof -i :8080 или sudo lsof -i TCP:8080. Вывод покажет COMMAND (имя процесса), PID, USER, тип соединения (IPv4/IPv6), DEVICE, SIZE/OFF, NODE и NAME (адреса и состояние). Права root нужны для просмотра процессов других пользователей.

8. Почему два процесса не могут одновременно слушать один и тот же TCP-порт?

Потому что при поступлении входящего TCP-пакета на этот порт операционная система (сетевой стек ядра) не сможет однозначно определить, какому из двух процессов его доставить. Это нарушило бы логику работы сетевого стека и привело бы к неопределённому поведению. Ограничение является защитным механизмом, предотвращающим конфликты между

сетевыми службами. Исключение — опция SO_REUSEPORT, позволяющая нескольким процессам слушать один порт при определённых условиях.

9. Как разрешить конфликт портов?

Существует два основных способа:

Завершить процесс, занимающий порт. Найти его PID через sudo ss -tlnp | grep <порт> или sudo lsof -i :<порт>, затем завершить командой kill <PID> (или kill -9 <PID> при необходимости).

Настроить новую службу на использование другого, свободного порта. Например, вместо порта 8000 использовать порт 8001.

Важно не завершать системные службы (sshd, systemd-resolved и т.п.), если не понятны последствия.

10. Что показывают эфемерные (динамические) порты в списке соединений?

Эфемерные порты (диапазон 49152–65535) — это временные порты, которые операционная система автоматически назначает клиентскому приложению при установлении исходящего соединения. Они используются на время сеанса связи и освобождаются после его закрытия. Например, при SSH-подключении к серверу на порт 22 клиентская сторона использует эфемерный порт (например, 55518), который виден в выводе ss как удалённый порт со стороны сервера.

11. Зачем при просмотре процессов, занимающих порты, часто нужны права root?

Без прав root утилита ss или lsof может показать только те процессы, которые принадлежат текущему пользователю. Системные службы (sshd, systemd-resolved, nginx и т.п.) обычно запускаются от имени root или специальных системных пользователей. Без sudo вместо имени процесса и PID будет отображаться прочерк (-), что делает диагностику невозможной.

12. Как отличить клиентский сокет от серверного (слушающего) в выводе ss?

Серверный (слушающий) сокет:

Состояние: LISTEN

Локальный порт — из диапазона well-known (0–1023) или registered (1024–49151), например, :22, :80, :8000

Удалённый адрес: 0.0.0.0:* или *:* (ожидает подключений от любого клиента)

Клиентский сокет:

Состояние: ESTABLISHED, TIME_WAIT, SYN_SENT и др. (но не LISTEN)

Локальный порт — эфемерный (49152–65535), например, :55518

Удалённый порт — известный (например, :22 для SSH, :80 для HTTP)